CAMPUSCLAW • 班级资料室

实现方案：基于 Token 的身份认证与基于知识库的问答

本文档对照仓库现有代码，逐层说明两条主链路的实现方式：**登录换取签名 token** 的认证链路， 以及 **入库切分 → 向量化 → 三种检索 → 带出处回答** 的知识库问答链路。 所有结论均来自当前代码与在运行的 Compose 环境实测，不包含设计构想。

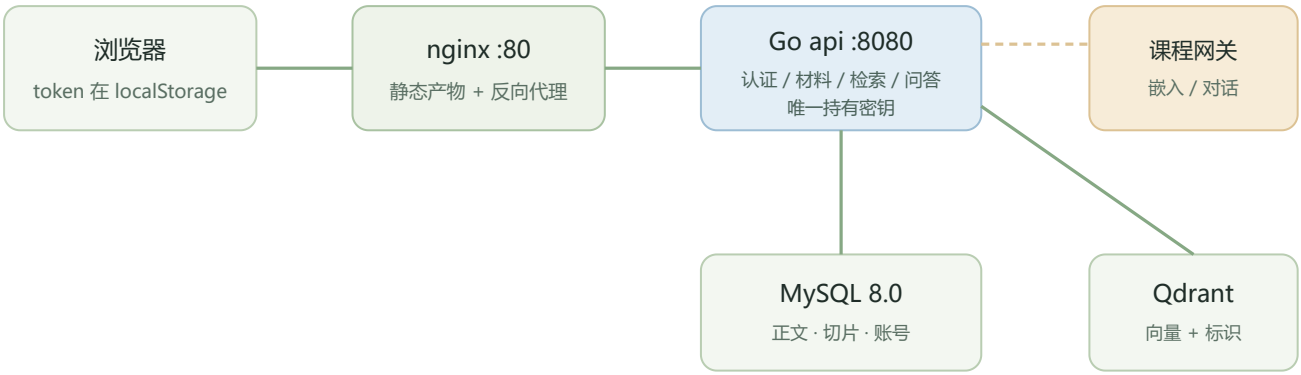
项目：CampusClaw（Go + MySQL 8.0 + Qdrant + React 18 / Vite + Nginx）
能力基线：`class-material-access` / `knowledge-qa` / `knowledge-retrieval`
变更：`switch-auth-to-jwt-token`、`add-class-scoped-qa`、`add-traceable-vector-retrieval`
日期：2026-09-30

目录

1. 系统组成与阅读指引
2. 基于 Token 的身份认证
   a. 目标与设计取舍
   b. 代码落点
   c. 令牌结构与签名算法
   d. 关键流程（签发 / 校验 / 自省 / 登出）
   e. 安全设计要点
   f. 接口契约与状态码
   g. 配置、密钥与部署
   h. 与第 3 课会话方案的对照
3. 基于知识库的问答
   a. 数据布局：正文与向量分离
   b. 入库链路：切分与索引状态机
   c. 切分策略与偏移基准
   d. 三模式检索与 RRF 融合
   e. 班级隔离的三重保障
   f. 问答链路与引用标注
   g. 失败与降级矩阵
   h. 接口契约
   i. 重建索引与历史材料补齐
   j. 前端视图
4. 两条链路的耦合点
5. 配置清单
6. 实测证据摘要
7. 已知限制与非目标

1. 系统组成与阅读指引

系统由四个容器组成：`mysql`（关系库，保存账号、班级、材料、正文与切片）、 `qdrant`（向量库，保存定长浮点向量与标识 payload）、`api`（Go 服务，唯一持有密钥与外部网关调用能力）、 `nginx`（托管前端静态产物并反代 `/api` 与 `/health`）。 只有 Nginx 绑定宿主机 `127.0.0.1:8080`，MySQL、Qdrant 与 API 均不发布宿主机端口， 客户端的唯一入口就是本站接口。



虚线 = 仅服务端调用，密钥不下发浏览器；MySQL / Qdrant / api 均不映射宿主机端口

图 1　系统组成与信任边界

阅读建议：第 2 章回答"我是谁、我在哪个班"，第 3 章回答"我能检索到什么、答案从哪来"。 两章之间的唯一致口是：**第 2 章确定的班级标识，正是第 3 章所有检索过滤条件的来源。**

2. 基于 Token 的身份认证

2.1 目标与设计取舍

认证需要同时满足四件事：登录后能识别账号；能识别账号所属班级（供材料与检索做范围隔离）； 能区分教师与学生（上传与重建索引只允许教师）；凭据不落 Cookie，浏览器只需持有它并在请求头里带上。 实现选择了**自签名的无状态 Bearer token（JWT 形态，HS256）**，理由是：

- 服务端无需保存会话记录，因此没有"会话表"这一层需要维护、清理或迁移的状态；
- 跨请求的身份由 token 自身承载，服务端只需持有唯一的签名密钥；
- 与旧方案相比，**不再需要 CSRF 防护**——凭据不再由浏览器自动携带，跨站请求无法"借用"身份。

> **取舍的代价（已在规约中明确）**：token 无状态，所以**登出只是客户端丢弃副本**，服务端没有可删除的记录， token 在到期前仍然可用；也正因为没有服务端状态，无法做单点强制下线或精细的令牌吊销。

2.2 代码落点

文件	职责
`internal/httpapi/token.go`	令牌签发（`issueToken`）、校验（`verifyToken`）、请求头解析（`bearerToken`）、鉴权中间件（`RequireIdentity`）、claims 结构
`internal/httpapi/login.go`	登录请求体严格解析、scrypt 口令校验、签发 token 并返回账号信息
`internal/httpapi/security.go`	`Security` 结构（数据库句柄 + 密钥 + 可注入时钟）、统一 JSON 响应与错误码
`internal/httpapi/session.go`	`GET /api/me`（回显当前身份）与 `POST /api/logout`（确认 token 有效）
`internal/httpapi/handler.go`	路由表与"哪些路径需要身份 / 哪些需要教师"的开关，以及业务就绪门禁
`internal/config/config.go`	`TOKEN_SECRET` 等必需项校验（缺失只报变量名，不打印值）
`frontend/src/api.ts`	token 的保存、自动附加 `Authorization` 头、401 时的清理规则

2.3 令牌结构与签名算法

令牌是紧凑 JWS（三段 base64url，点号分隔）：`base64url(header) . base64url(payload) . base64url(HMAC-SHA256(前两段))`。

```
header（常量，不接受任何其他取值）
{"alg":"HS256","typ":"JWT"}

payload（claims）
{"sub":3,"username":"student_b1","role":"student","classId":2,"iat":1759200000,"exp":1759228800}
  sub      账号主键，requireIdentity 用它回查账号表
  username 账号名（展示用）
  role     角色（展示用；鉴权一律以数据库为准）
  classId  班级（展示用；过滤一律以数据库为准）
  iat/exp  签发与到期时间戳，有效期 8 小时（tokenLifetime = 8h）

签名
HMAC-SHA256(base64url(header) + "." + base64url(payload), TOKEN_SECRET)
```

实现细节：header 被写死为常量字符串并在校验时逐字节比对；base64 使用 `RawURLEncoding.Strict()`（拒绝非法填充与多余字符）； 签名比较使用 `hmac.Equal`；claims 解码要求"恰好一个 JSON 值"（有多余内容即失败）。

2.4 关键流程

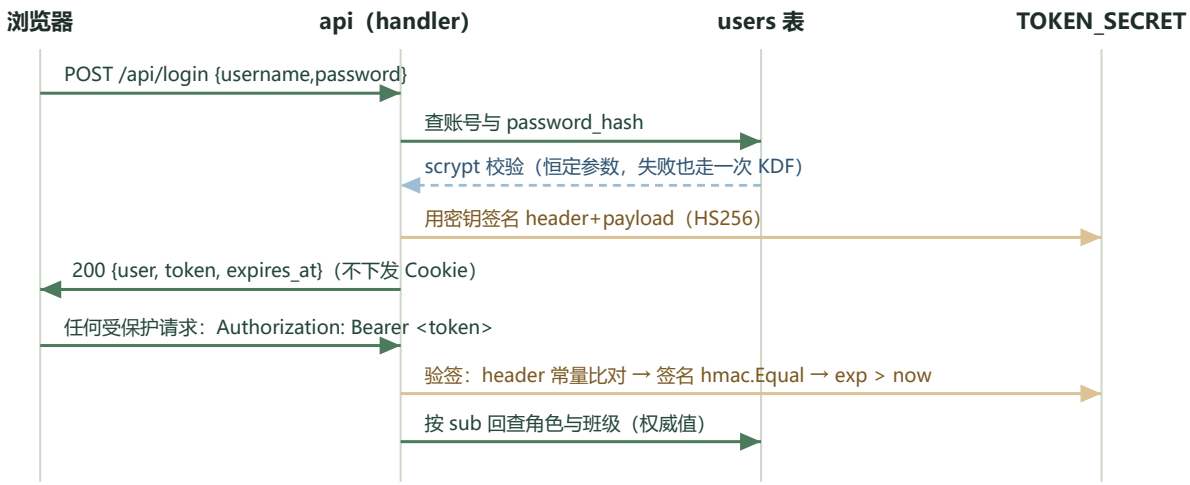


图 2　签发与校验时序

（1）登录签发 —— `POST /api/login`

1. **严格解析请求体**：只接受 `application/json`；长度超过 16 KiB、非 UTF-8、不是单个 JSON 对象、 出现重复字段或尾随内容一律拒绝；`username` 长度 1–64 且非纯空白，`password` 长度 1–128。
2. **查账号**：`SELECT id,username,role,class_id,password_hash FROM users WHERE username=?`（5 秒超时）。 查询出错（非"无记录"）返回 503，避免把数据库故障说成密码错误。
3. **校验口令**：密码哈希格式为 `$scrypt$32768$8$1$<salt>$<key>`； KDF 参数写死在代码里（不使用数据库字符串里的参数），盐 16 字节、派生键 32 字节， 比较用 `hmac.Equal`。未知用户同样执行一次 KDF，使"账号不存在"与"口令错误"耗时接近。
4. **签发**：以 `sub/username/role/classId/iat/exp` 组装 claims，用 `TOKEN_SECRET` 做 HMAC-SHA256 签名， 返回 `{user, token, expires_at}`；**不设置任何 Cookie。**

请求体里的 `role`、`class_id` 等字段根本不会被读取——身份只来自账号表。

（2）请求校验 —— `RequireIdentity`

所有受保护路由都包在这个中间件里，顺序固定如下：

1. **取 token**：`Authorization` 头必须**恰好一条**，前缀必须是 `Bearer`， token 非空且不含空格或制表符；否则 401。
2. **验签**：三段结构 → header 逐字节等于常量 `{"alg":"HS256","typ":"JWT"}` → 签名长度等于 32 字节且 `hmac.Equal` 通过 → claims 可解析且 `sub != 0`、`exp > now`； 任一环节不符都返回同一个 401，不区分原因。
3. **回查权威身份**：`SELECT id,username,role,class_id FROM users WHERE id=<sub>`。 无记录 → 401（账号已被删除时旧 token 立即失效）；数据库错误 → 503。
4. **角色门禁**：`teacherOnly` 的路由（上传材料、重建索引）要求 `role == "teacher"`， 否则 403。
5. **注入上下文**：把账号行放入 `context`（键 `identityKey{}`）， 下游处理器只从这个值读取身份与班级。

> **关键点：**token 里的 `role` 与 `classId` **只用于展示。** 授权与范围过滤永远使用第 3 步回查到的数据库值，因此即使有人伪造/篡改 token 载荷也无法越权—— 签名不过就进不来，签名过得来则意味着密钥已经泄露，此时载荷本身也不构成额外风险。

（3）身份自省与登出

- `GET /api/me`：中间件已验签并回查账号，处理器直接把上下文里的账号行返回； **不会把 token 原样回显。**
- `POST /api/logout`：只确认所出示 token 有效，然后返回 `{"ok":true}`。 由于令牌无状态，真正的"退出"是客户端丢弃本地副本。

（4）业务就绪门禁

除 `/health` 之外的所有路径，在服务端"业务就绪"标志为假时直接返回 503。 就绪标志由启动时的恢复流程设置：先探测业务表可查询、再清理上传暂存与孤儿文件，全部成功才置真。 因此 `/health` 只表示"进程存活"，**不代表业务可用**——这一点在排障时非常关键。

2.5 安全设计要点

风险	对策	代码位置
alg=none / 算法混淆攻击	不解析 header 里的算法，而是要求 header 字符串**逐字节等于**固定常量；任何其他 header 直接失败	`token.go: tokenHeader`、`verifyToken`
签名比较被计时侧信道推断	使用 `hmac.Equal` 常量时间比较，并先校验签名长度	`token.go: verifyToken`
利用请求头歧义绕过鉴权	Authorization 头必须恰好一条、只能是 Bearer 方案、token 内不允许空白字符	`token.go: bearerToken`
客户端伪造角色或班级	claims 仅作展示；授权与过滤一律以账号表回查值为准；登录请求体中的 role/class_id 被忽略	`token.go: RequireIdentity`

账号被删除后旧令牌继续可用	每个请求都回查账号行，查不到即 401	`token.go: RequireIdentity`
口令被离线爆破 / 参数被 SQL 数据左右	scrypt 参数固定（N=32768, r=8, p=1, dkLen=32）；未知用户也执行一次 KDF；恒定时间比较	`login.go: passwordMatches`
解析歧义（重复字段、类型混淆、超大体积）	登录与提问等接口都手写严格解析：限长、限 UTF-8、单对象、禁止重复键、拒绝尾随内容	`login.go: parseLogin`、`ask.go: parseAsk`、`search.go: parseReindex`
CSRF	凭据不走 Cookie，浏览器不会自动携带，跨站请求无法冒用身份；因此不再需要 CSRF 令牌	设计层面（无对应代码）
凭据泄漏到日志 / 前端产物	配置错误只报变量名；前端构建脚本 `check-dist.mjs` 会扫描产物中是否出现服务端凭据名与值	`config.go`、`frontend/scripts/check-dist.mjs`

2.6 接口契约与状态码

方法与路径	鉴权	成功响应	失败
`POST /api/login`	匿名	200 `{user:{id,username,role,class_id}, token, expires_at}`	400 `invalid_request`；401 `invalid_credentials`；503
`GET /api/me`	需身份	200 `{id,username,role,class_id}`	401 `unauthenticated`；503
`POST /api/logout`	需身份	200 `{"ok":true}`	401；503
`GET /api/materials` 等材料读取	需身份	200	401；404 `not_found`（跨班与不存在同为 404）；503
`POST /api/materials`	需身份 + 教师	201 `{"material_id":…}`	400；401；403 `forbidden`；413；500 `internal_error`；503
`POST /api/materials/{id}/reindex`	需身份 + 教师	200 `{material_id,strategy,chunks}`	400；401；403；404；503
`GET /api/search`	需身份	200 `{mode,hits[]}`	400；401；503
`POST /api/ask`	需身份	200 `{answer,hits[],citations[]}`	400；401；503
`GET /health`	匿名	200 `{"status":"ok"}`	—

统一错误体为 `{"error":"<code>","message":"<安全文案>"}`；响应统一带 `Cache-Control: no-store`。旧方案的 `GET /api/csrf` 已删除，访问该路径现在是普通 404。

2.7 配置、密钥与部署

- 唯一的认证密钥是环境变量 `TOKEN_SECRET`（Compose 中为必填，缺失时配置校验直接失败）。注入方式为进程环境，不写入源码、不打进前端产物、不落日志。
- 有效期固定 8 小时、不滑动续期，也没有"刷新令牌"；到期后重新登录。
- 迁移 `000002_drop_sessions` 删除了第 3 课的 `sessions` 表，因此当前业务表为：`classes`、`users`、`materials`、`knowledge_entries`、`knowledge_chunks`。
- 密钥轮换等价于"全体重新登录"；更换 `TOKEN_SECRET` 后所有未过期 token 立即失效。

2.8 与第 3 课会话方案的对照

维度	第 3 课：Cookie 会话	本次：无状态 Token
凭据载体	`HttpOnly` Cookie	`Authorization: Bearer <jwt>`，浏览器存 `localStorage`
服务端状态	需要 `sessions` 表（含过期时间、哈希）	无状态，表已删除
CSRF	必需（写操作校验 `X-CSRF-Token` + Origin）	不需要，相关代码与接口已移除
登出语义	删除会话记录，立即失效	客户端丢弃；服务端没有可删除的记录，到期前仍可用
身份权威来源	会话行 JOIN 用户表	token 验签后用 `sub` 回查用户表
班级/角色来源	会话关联的账号行	账号表回查值（claims 仅供展示）

3. 基于知识库的问答

3.1 数据布局：正文与向量分离

设计原则是"正文永远在关系库，向量库只放向量与标识"。这样做的直接好处是：摘录永远可追溯到原文、 切片与材料的事务一致性可以依赖 MySQL 的外键与约束、更换嵌入模型只需重建向量而不动正文。

位置	保存内容	说明
MySQL `knowledge_entries.body_text`	上传原文	切分与预处理都不改写它
MySQL `knowledge_chunks`	`chunk_text` 、 `char_start` 、 `char_end` 、 `chunk_index` 、 `strategy` 、 `index_status` 、 `knowledge_entry_id`	切片正文与字符区间；带 `FULLTEXT … WITH PARSER ngram` 全文索引
Qdrant `campusclaw_chunks`	定长浮点向量 + payload `{class_id, material_id, knowledge_entry_id, chunk_id, chunk_index}`	Cosine 距离，维度等于 `EMBEDDING_DIM` ；**不存正文**

三重标识保持同值：Qdrant 的点主键、 `knowledge_chunks.id` 与 `payload.chunk_id` 。 因此向量检索命中后可以直接按主键回表取正文，不需要在向量库里存一份正本。

3.2 入库链路：切分与索引状态机

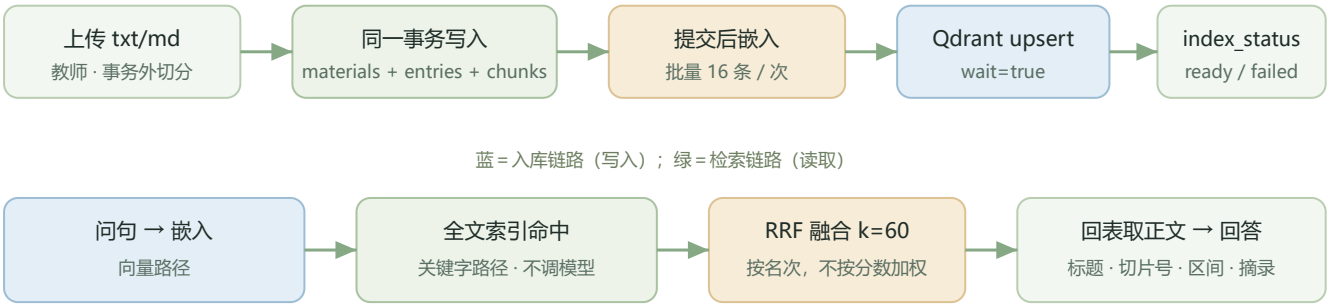


图 3　入库与检索两条链路

入库的六步实现：

1. **切分在事务之外**：上传的正文先在内存里预处理并切分；参数非法或正文不可切分时直接失败，连事务都不开，天然不留半成品。
2. **材料 + 正文 + 切片同事务提交**： `materials` 、 `knowledge_entries` 、 `knowledge_chunks` 三条写入在一个事务里，任一失败整体回滚，并清理已落盘的文件。
3. **提交后再嵌入**：调用嵌入网关（每批 16 条）拿到向量，再以切片主键为点主键写入 Qdrant。 这一步在事务之外，因为它是外部调用，把它放进事务会长时间占用连接并放大失败面。
4. **状态标注**：切片默认 `pending` （表示"正文已入库、向量未写"）； 嵌入与写入都成功置 `ready` ，任一步失败置 `failed` 。 若根本没有配置向量库，写入时就直置 `ready` ——关键字索引在那一刻已经可用。
5. **失败不毁数据**：嵌入失败不回滚材料，材料与正文保持有效，只是对应切片标为失败；检索两路都只取 `ready` 的切片，因此不会把不完整的向量带进答案。
6. **可修复**：教师对该材料发起一次显式重建即可恢复（见 3.9）。

3.3 切分策略与偏移基准

策略	参数	断点规则
`auto` （默认）	固定 800 字节、重叠 80 字节	优先在空行断开，其次换行，再次句末标点；切点限制在窗口四分之一之后，避免产生极短切片
`custom`	`size` 100–2000； `overlap ≤ size/2` （即 0%–50%）	同 auto；可选 `remove_urls` 、 `remove_emails` 、 `collapse_whitespace`
`hierarchy`	无参数	按 Markdown `#` / `##` / `###` 分章，标题保留在章内；单章超过窗口时再按 auto 窗口细分

偏移基准必须说清楚： `char_start` / `char_end` 是预处理后文本的字节区间。 未启用预处理时预处理文本就等于原文，区间可直接用来截取正文逐字复现；启用移除链接/邮箱或折叠空白后， 基准变成预处理结果，此时区间仍能复现切片文本，但不再是原始文件的下标。切片、字段注释与接口文档都按这个口径表述。

另外：切片文本在统一前会丢弃纯空白片段，并保证相邻切片的区间严格递增（数据库层用 `CHECK (char_end > char_start)` 兜底）。

3.4 三模式检索与 RRF 融合

模式	执行内容	依赖
`keyword`	只查 MySQL 全文索引： `WHERE class_id = 会话班级 AND index_status='ready' AND MATCH(chunk_text) AGAINST(问句) > 0` ， 按全文相关度降序、切片主键升序	仅 MySQL；**不调用嵌入网关、不访问向量库**

`vector`	先把问句嵌入，再在 Qdrant 按 `class_id` 过滤做余弦检索，丢弃相似度低于 **0.35** 的候选， 最后按主键回表取正文	嵌入网关 + Qdrant
`hybrid`（缺省）	两条路径各自取前 20 条候选，再按**名次**做 RRF 融合取前 N 条（检索接口默认 4 条，可传 `limit` 1–20）	两者；严格模式下任一路不可用即 503

RRF（Reciprocal Rank Fusion）的计分方式：

```
score(chunk) = Σ_paths  1 / (k + rank)          k = 60，rank 从 1 开始
  · 只在某一路出现的切片，另一路不贡献分数
  · 两路都排在前面的切片必然得分最高（例：两路都排第 1 → 1/61 + 1/61 ≈ 0.032787）
  · 名次分只是排序依据，不是相关性阈值，也不跨模式之间比较
```

为什么用名次而不是分数加权：向量余弦与全文相关度量纲不同，归一化后加权需要针对具体嵌入模型重新标定权重； 名次融合只依赖顺序，更换嵌入模型时不需要重新调参。

响应里每条命中都会带 `score`：关键字模式是全文相关度、向量模式是余弦相似度、混合模式是 RRF 名次分， 用于展示排序依据，不作为对外承诺的阈值。

3.5 班级隔离的三重保障

1. **身份来源唯一**：班级来自 `RequireIdentity` 回查账号表得到的 `class_id`； 查询串、请求头、请求体里的 `class_id` 从头到尾不会被读取。
2. **两条路径各自过滤**：关键字路径在 SQL 的 `WHERE` 里带班级；向量路径在 Qdrant 的 `filter.must` 里带班级。 即使其中一侧漏了条件，另一侧仍会排除其他班级。
3. **回表再核对**：以主键回 MySQL 取正文时，SQL 再次带上 `class_id` 与 `index_status='ready'`， 因此向量库里的越界候选不会进入结果。

> **对外表现：**跨班检索返回 **HTTP 200 与空命中**（问答则返回"资料中未找到相关内容"）， 刻意不使用 403/404——那两个状态会向调用方暗示"这份资料存在但不属于你"。 只有按材料标识打开详情或原文时沿用既有规则返回同形 404（跨班与不存在不可区分）。

3.6 问答链路与引用标注

`POST /api/ask` 的处理顺序是固定的"先检索、后生成"：

1. **解析**：只接受单个 JSON 对象与唯一的 `question` 字段（限长 500 字）；客户端传来的 `system` 消息与历史轮次一概忽略。系统提示由服务端自己写入。
2. **检索**：以 `hybrid` 模式取本班前 **4** 条切片。
3. **无依据则不生成**：候选为空时直接返回固定文案「资料中未找到相关内容。」， `hits` 与 `citations` 为空数组，**完全不调用对话网关**，也不使用模型自身记忆作答。
4. **有依据才生成**：把每条切片对应的材料标题、切片序号、切片正文与用户本轮提问组成提示， 要求回答简短并标注出处；回答里的 `[1]`、`[2]` 与返回数组顺序一一对应。
5. **非流式**：返回完整 JSON，不做分片输出（流式属于下一课）。

```
200 {
  "answer": "磁铁不能吸引木块。[1]",
  "hits": [ { "material_id":5, "title":"小学科学课程资料.md", "chunk_id":9, "chunk_index":1,
              "char_start":544, "char_end":905, "excerpt":"…切片正文…", "citation":1 } ],
  "citations": [ { "material_id":5, "title":"小学科学课程资料.md", "chunk_id":9, "chunk_index":1 } ]
}
```

`hits` 是可回溯的完整摘录（供界面展示出处）， `citations` 是同一顺序的紧凑标识（供程序对齐 `[n]` 标记）。两者顺序严格一致，且摘录一律取自 MySQL 的 `chunk_text`。

3.7 失败与降级矩阵

同一套检索实现，两个入口对"路径故障"的处置**刻意不同**： 检索接口要求"混合就是混合"，任一在用路径不可用就如实报错；问答接口允许在关键字已能作答时继续，避免依赖故障把功能整体打断。

场景	`GET /api/search`	`POST /api/ask`
未配置 `EMBEDDING_*`	`keyword` 正常； `vector` / `hybrid` → 503	自动只用关键字路径，照常作答；无候选则返回固定文案
Qdrant 停机	`keyword` 正常； `vector` / `hybrid` → 503	关键字已有候选 → 继续作答；两路都无候选 → 503
嵌入网关不可达	`vector` / `hybrid` → 503	同上（按关键字候选是否存在决定 503 或继续）
对话网关不可用	不涉及	有命中 → 503，且不返回编造的回答
入库时嵌入失败	该切片 `failed`，不被检索（其余材料不受影响）	同左；材料与正文仍保留，可重建恢复
空查询 / 非法模式	400 `invalid_request`	400（空问题、非 JSON、超长）
向量库不可用时启动服务	进程照常启动（只记一行日志）， `/health` 200，向量相关请求 503	

3.8 接口契约

接口	参数 / 请求体
`GET /api/search`	`q`（必填，≤500 字）、 `mode`（`keyword|vector|hybrid`，缺省 hybrid）、 `limit`（1–20，缺省 4）

`POST /api/ask`	`{"question":"…"}`
`POST /api/materials/{id}/reindex`	`{"strategy":"auto|custom|hierarchy","size":…,"overlap":…,"remove_urls":…,"remove_emails":…,"collapse_whitespace":…}`（整体可省略，缺省即 auto）

3.9 重建索引与历史材料补齐

重建是**显式操作**，不会因为改了策略就自动重切分——这是为了防止"改一个参数导致全量重算"的隐性开销。执行顺序是：

1. 按 `material_id` 删除该材料在 Qdrant 里的全部向量（用 payload 过滤，而不是点 id 列表，避免漏删已不存在的旧切片）；
2. 在同一事务里删除旧切片行、按本次策略写入新切片（`strategy` 记录实际使用的策略）；
3. 提交后重新嵌入并写回 Qdrant；失败只标 `failed`，材料与正文不变，需要人工再次重建（不自动重试）。

第 3 课入库的历史材料没有切片，需要一次 backfill 补齐：命令 `cmd/reindex`（容器内为 `/app/reindex`）， 可传材料 id，省略即全部材料，统一按 `auto` 策略重建。

3.10 前端视图

视图	能力
班级资料	本班材料列表、详情预览、下载；教师可上传、可按 `auto` / `hierarchy` / `custom` 重建当前材料索引
知识检索	一句自然语言 + 模式选择（默认混合）；命中卡片展示材料标题、切片序号、字符区间、排序分与摘录，并可一键跳转到对应材料
知识问答	对话流展示回答与逐条引用（材料标题 / 切片号 / 字符区间 / 摘录）；无依据时显示固定文案

前端不参与任何过滤或排序：模式、班级与命中范围全部由服务端决定，浏览器只负责展示与携带 token。

4. 两条链路的耦合点

认证与检索不是两个孤立模块，它们的接口只有一处，但这一处决定了全部数据可见范围：

1. **班级的唯一来源**：token 验签 → `sub` → `users.class_id`。 材料列表、材料详情、附件下载、检索、问答、重建索引，全部使用同一个值做范围过滤。
2. **角色的唯一来源**：同一个 `userIdentity` 决定了能否上传、能否重建索引（`teacherOnly`）。
3. **不可绕过**：因为班级既不来自请求参数、也不单独信任 token 载荷，所以"伪造 class_id 去检索别的班" 在实现上无处生效——参数根本没被读取。

5. 配置清单

变量	作用	必填
`TOKEN_SECRET`	JWT（HS256）签名与验签密钥	是（缺失即启动失败，只报变量名）
`DB_HOST` / `DB_PORT` / `DB_NAME` / `DB_USER` / `DB_PASSWORD`	应用数据库连接	是
`UPLOAD_DIR`	材料原文件目录（Compose 内为 `/data/uploads`）	是（必须为绝对路径）
`HTTP_ADDR`	监听地址，缺省 `127.0.0.1:8080`	否
`SEED_TEACHER_A_PASSWORD` 等三个	仅初始化命令使用，用于生成账号口令哈希	仅 `cmd/seed`
`QDRANT_URL` / `QDRANT_COLLECTION`	向量库地址与集合名（缺省 `http://qdrant:6333`、`campusclaw_chunks`）	否
`EMBEDDING_BASE_URL` / `EMBEDDING_API_KEY` / `EMBEDDING_MODEL` / `EMBEDDING_DIM`	嵌入网关；四项齐全才启用向量路径，`DIM` 用于校验返回向量长度	否（缺失即关闭向量能力）
`CHAT_BASE_URL` / `CHAT_API_KEY` / `CHAT_MODEL`	对话网关，用于生成带出处的回答	否
`MYSQL_ROOT_PASSWORD` / `CAMPUSCLAW_PORT`	仅 Compose：MySQL root 口令、Nginx 本地端口	仅部署

固定策略（非开关）：token 有效期 8 小时；切分 auto = 800/80；向量候选阈值 0.35；RRF `k=60`；问答取前 4 条。 以上常量写在代码里，避免"改环境变量即改变行为"的隐式风险。

6. 实测证据摘要

以下结果来自在运行的 Compose 环境（四容器 healthy）与真实浏览器，均可复现。

链路	验证项	实测结果
身份认证	无 token 访问受保护接口	401 `unauthenticated`；`GET /api/csrf` 为 404
	登录	返回 `user/token/expires_at`，`Set-Cookie` 数量为 0
	token 载荷	`{"sub":3,"username":"student_b1","role":"student","classId":2,…}`，`exp-iat=28800`
	带 token 访问	`/api/me`、`/api/materials` 均 200
	篡改 / 降级算法 / 非 Bearer 方案	签名被改、`alg=none`、`Basic` 三种均为 401
	角色与范围	学生上传 → 403；A 班账号读 B 班材料详情与原文 → 404
知识库问答	切片与偏移	5 条切片与 Qdrant 5 个点一一对应；`auto` 得 `0-622`、`544-905`（重叠 78 字节），按字节可复现正文
	三种模式	同一问句：keyword 1.109、vector 0.4277、hybrid 0.032787（= 2/61）；不传 `mode` 即 hybrid
	跨班检索	B 班检索 A 班独有词并伪造 `class_id=1` → 200 且 `hits=0`
	参数校验	空查询 400、非法模式 400、非法 `custom` 参数 400 且不动切片
	重建索引	`hierarchy` 得 4 片且标题在章内；正文 MD5 前后一致；旧切片与旧向量零残留
	无依据不生成	材料外问题 → 200 固定文案、`hits:[]`；把对话网关指向不可达地址后该问题仍 200（对照：材料内问题 503）
	依赖降级	停 Qdrant：keyword 200、vector/hybrid 503；api 仍能启动，`/health` 200，日志仅一行提示
工程校验	`go build` / `go vet` / `go test ./…` 全绿；前端 `tsc` + `vite build` + 产物密钥扫描通过；浏览器实跑三模式无运行时错误	

7. 已知限制与非目标

项	说明
无令牌吊销	令牌无状态，登出只清除客户端副本；到期前令牌仍有效。没有黑名单、没有强制下线。
无刷新令牌	有效期固定 8 小时、不滑动续期，到期需重新登录。
无流式回答	`/api/ask` 返回完整 JSON（流式属于后续课程）。
无重排序 / 无分数加权	只用 RRF 名次融合，未引入交叉编码器重排或按模型标定的加权求和。
不引入编排框架	切分、融合、网关与向量库客户端全部用 Go 标准库实现，未引入 LangChain / LlamaIndex 一类依赖。
不自动重切分	改变切分策略必须显式重建索引；未重建的材料保持原切片。
班级不跨域共享	不存在"跨班检索"或管理员越权查看；对外一律表现为空命中。
更细粒度的权限模型	当前只有教师/学生两种角色，没有按材料级的授权。